

MODUL 5 INTRRUPT DAN TIMER

Kaira Meilasya Nayada (H1H024020)

Asisten: Muhammad Nur Bijak B

Tanggal Percobaan: 18/05/2026

TK244005-Praktikum Sistem Mikrokontroler

Laboratorium Multimedia – Jurusan Informatika UNSOED

Abstrak

Percobaan pada modul ini bertujuan untuk memahami dan mengimplementasikan mekanisme interrupt dan timer pada mikrokontroler Arduino Uno. Percobaan pertama (6A) menggunakan external interrupt berbasis push button untuk mengendalikan kondisi LED tanpa metode polling. Percobaan kedua (6B) mengimplementasikan LED blinking menggunakan fungsi millis() sebagai pengganti delay() sehingga sistem tetap responsif. Metode yang digunakan meliputi pengkabelan komponen sesuai tabel konfigurasi pin, penulisan ISR (Interrupt Service Routine) menggunakan attachInterrupt(), penggunaan variabel volatile, serta teknik non-blocking timer berbasis millis(). Hasil menunjukkan bahwa kedua percobaan berhasil dijalankan sesuai spesifikasi yang ditetapkan, dengan LED yang merespons interrupt tombol secara langsung dan berkedip secara periodik tanpa menghambat proses utama mikrokontroler.

Kata kunci: Arduino Uno, LED, ISR, Timer, millis(), Non-blocking, Mikrokontroler.

1. PENDAHULUAN

Sistem tertanam (embedded system) modern memerlukan kemampuan merespons kejadian eksternal secara cepat dan efisien. Salah satu keterbatasan metode polling yang umum digunakan adalah penggunaan CPU yang tidak optimal, karena mikrokontroler terus-menerus memeriksa kondisi input meskipun tidak terjadi perubahan [1]. Untuk mengatasi hal tersebut, mikrokontroler seperti Arduino Uno menyediakan mekanisme interrupt dan timer yang memungkinkan penanganan kejadian secara asinkron[1].

Pada praktikum kali ini dilakukan dua percobaan. Percobaan pertama (6A) berfokus pada implementasi external interrupt menggunakan push button untuk mengubah kondisi LED secara langsung melalui ISR. Percobaan kedua (6B) mengimplementasikan timer berbasis fungsi millis() untuk menghasilkan LED blinking dengan interval 1 detik tanpa menggunakan delay(), sehingga sistem tetap dapat menjalankan tugas lain secara bersamaan [2].

Laporan ini memuat uraian dasar teori, metodologi pelaksanaan, hasil percobaan beserta analisis, serta kesimpulan yang diperoleh dari seluruh rangkaian kegiatan yang telah dilakukan.

2. STUDI PUSTAKA

2.1 INTERRUPT

Interrupt adalah mekanisme pada mikrokontroler yang memungkinkan penghentian sementara eksekusi program utama untuk menangani suatu kejadian tertentu. Ketika interrupt terjadi, mikrokontroler menyimpan konteks program yang sedang berjalan, kemudian menjalankan fungsi khusus bernama Interrupt Service Routine (ISR). Setelah ISR selesai dieksekusi, program utama dilanjutkan kembali dari titik yang sama [1].

Arduino Uno (berbasis ATmega328P) menyediakan dua jalur external interrupt, yaitu pada pin 2 (INT0) dan pin 3 (INT1). Pendaftaran ISR dilakukan menggunakan fungsi attachInterrupt() dengan format: attachInterrupt(digitalPinToInterrupt(pin), ISR, mode). Mode pemicu yang tersedia meliputi RISING (tepi naik), FALLING (tepi turun), CHANGE (setiap perubahan), dan LOW (kondisi rendah) [3].

2.2 INTERRUPT SERVICE ROUTINE (ISR)

SR adalah fungsi callback yang dieksekusi secara otomatis saat interrupt terjadi. Prinsip utama dalam penulisan ISR adalah membuatnya sesingkat dan sesederhana mungkin, karena selama ISR berjalan proses utama ditangguhkan. Fungsi delay() dan Serial.print() tidak boleh digunakan dalam ISR karena keduanya bergantung pada mekanisme timer dan interrupt yang sedang terblokir [1].

2.3 VARIABEL VOLATILE

Keyword volatile digunakan pada variabel yang diakses bersama oleh ISR dan program utama. Tanpa keyword ini, compiler dapat melakukan optimasi yang menyebabkan nilai variabel tidak diperbarui secara benar karena dianggap tidak berubah di luar alur normal program. Penggunaan volatile memaksa compiler untuk selalu membaca nilai terbaru dari memori [4].

2.4 TIMER DAN FUNGSI MILLIS()

Timer adalah modul internal mikrokontroler yang menghitung waktu berdasarkan clock sistem. Arduino Uno memiliki tiga modul timer internal: Timer0 (8-bit), Timer1 (16-bit), dan Timer2 (8-bit). Fungsi `millis()` memanfaatkan Timer0 untuk mengembalikan jumlah milidetik sejak Arduino pertama kali dinyalakan [2].

2.5 METODE NON-BLOCKING DENGAN MILLIS()

Berbeda dengan `delay()` yang memblokir eksekusi seluruh program, metode `millis()` bersifat non-blocking. Pola penggunaannya adalah dengan menyimpan waktu terakhir suatu aksi dilakukan, kemudian membandingkannya dengan waktu saat ini pada setiap iterasi `loop()`. Jika selisihnya telah melampaui interval yang diinginkan, aksi dijalankan dan waktu referensi diperbarui [5]. Keunggulan utama metode ini adalah kemampuannya menjalankan beberapa tugas secara pseudo-paralel, menjadikannya cocok untuk implementasi multitasking sederhana pada sistem embedded [2].

3. METODOLOGI

3.1 ALAT DAN BAHAN

Berikut adalah komponen yang digunakan pada seluruh percobaan:

- Arduino Uno (berbasis ATmega328P)
- Push Button (1 buah)
- LED (2 buah)
- Resistor 220 Ω (2 buah untuk LED)
- Resistor 10k Ω (1 buah untuk pull-down tombol)
- Breadboard
- Kabel Jumper
- Komputer dengan Arduino IDE / PlatformIO

3.2 PERCOBAAN 6A: EXTERNAL INTERRUPT

Percobaan ini bertujuan mengendalikan LED menggunakan external interrupt berbasis push button tanpa polling. Rangkaian dibuat dengan menghubungkan LED ke pin 13 dan push button ke pin 2 Arduino menggunakan breadboard sesuai Tabel 3.1.

No	Komponen	Pin Arduino
1	LED	Pin 13
2	Push Button	Pin 2
3	GND	GND
4	VCC	5V

Tabel 3.1. Konfigurasi Wiring Percobaan 6A

Program dikonfigurasi dengan mendeklarasikan variabel volatile bool `ledState`, mendaftarkan ISR melalui `attachInterrupt()` pada pin 2 dengan mode `FALLING`, dan pada setiap iterasi `loop()` menuliskan nilai `ledState` ke pin 13. ISR hanya melakukan toggle nilai `ledState` saat tombol ditekan.

3.3 PERCOBAAN 6B: TIMER MENGGUNAKAN MILLIS()

Percobaan ini bertujuan membuat LED blinking dengan interval 1 detik menggunakan metode non-blocking `millis()` tanpa `delay()`. Rangkaian menggunakan satu LED pada pin 13 sesuai Tabel 3.2.

No	Komponen	Pin Arduino
1	LED	Pin 13
2	GND	GND

Tabel 3.2. Konfigurasi Wiring Percobaan 6B

Variabel `previousMillis` menyimpan waktu terakhir LED berubah, sementara interval ditetapkan 1000 ms. Setiap iterasi `loop()`, program mengambil waktu saat ini menggunakan `millis()`, kemudian mengecek selisihnya terhadap `previousMillis`. Jika selisih telah melampaui interval, `ledState` di-toggle dan `previousMillis` diperbarui.

4. HASIL DAN ANALISIS

4.1 HASIL PERCOBAAN 6A: EXTERNAL INTERRUPT

Setelah program diunggah ke Arduino, LED pada pin 13 berhasil berubah kondisi ON/OFF setiap kali push button ditekan. Tidak terjadi pembacaan ganda maupun keterlambatan respons yang berarti, menunjukkan mekanisme interrupt bekerja dengan baik. Sistem juga tidak melakukan pengecekan kondisi tombol secara berulang pada `loop()` utama,

membuktikan bahwa polling telah digantikan sepenuhnya oleh interrupt.

Percobaan berjalan sesuai seluruh spesifikasi yang ditetapkan: pin telah dikonfigurasi sesuai petunjuk, LED berubah kondisi saat tombol ditekan, dan sistem tidak menggunakan polling. ISR yang dibuat sangat ringkas (hanya satu baris toggle) sehingga tidak mengganggu kinerja program utama.

4.2 PERTANYAAN PERCOBAAN 6A

1. Proses interrupt mengubah kondisi LED: Ketika tombol ditekan, sinyal pada pin 2 mengalami transisi dari HIGH ke LOW (FALLING edge) karena pin dikonfigurasi dengan INPUT_PULLUP. Transisi ini memicu interrupt INT0, yang menyebabkan mikrokontroler menghentikan sementara eksekusi loop() dan langsung menjalankan fungsi tombolInterrupt(). ISR tersebut melakukan toggle pada variabel ledState. Setelah ISR selesai, loop() dilanjutkan dan digitalWrite() pada iterasi berikutnya menuliskan nilai ledState terbaru ke LED.

2. Fungsi attachInterrupt(): Fungsi ini mendaftarkan hubungan antara pin interrupt fisik dengan fungsi ISR yang akan dipanggil, serta menentukan kondisi sinyal pemicunya (mode). Tanpa fungsi ini, mikrokontroler tidak mengetahui ISR mana yang harus dieksekusi saat interrupt terjadi.

3. Alasan delay() dan Serial.print() dilarang dalam ISR: Fungsi delay() bergantung pada interrupt Timer0 yang sedang terblokir selama ISR berjalan, sehingga akan menyebabkan deadlock. Serial.print() menggunakan buffer berbasis interrupt UART yang juga tidak dapat diproses. Keduanya dapat menyebabkan sistem hang atau berperilaku tidak terduga.

4. Fungsi keyword volatile: Keyword volatile memberitahu compiler bahwa nilai variabel ledState dapat berubah di luar alur normal program (yaitu di dalam ISR). Tanpa volatile, compiler dapat menyimpan nilai variabel dalam register CPU dan tidak membaca ulang dari memori, sehingga perubahan yang dilakukan ISR tidak terlihat oleh loop() utama.

5. Perbedaan mode interrupt: Mode RISING memicu interrupt saat sinyal berubah dari LOW ke HIGH (tombol dilepas pada konfigurasi pull-up). Mode FALLING memicu saat sinyal berubah dari HIGH ke LOW (tombol ditekan). Mode CHANGE memicu pada setiap perubahan sinyal, baik naik maupun turun, sehingga satu penekanan tombol memicu dua kali interrupt. Mode LOW terus memicu selama pin berada pada kondisi LOW,

berpotensi menyebabkan ISR dipanggil berulang kali selama tombol ditahan. Pada percobaan, mode FALLING paling tepat digunakan karena hanya merespons saat tombol pertama kali ditekan.

4.3 HASIL PERCOBAAN 6B: TIMER MENGGUNAKAN MILLIS()

LED pada pin 13 berhasil berkedip dengan interval tepat 1 detik secara konsisten. Program tidak menggunakan delay() sehingga CPU tetap dapat menjalankan instruksi lain pada setiap iterasi loop(). Hal ini dibuktikan dengan kemampuan sistem merespons input lain secara bersamaan jika ditambahkan pada program.

Seluruh spesifikasi percobaan terpenuhi: LED berkedip setiap 1 detik, program menggunakan millis() tanpa delay(), dan sistem tetap responsif selama LED berkedip.

4.4 PERTANYAAN PERCOBAAN 6B

1. Cara kerja millis() pada program: Setiap iterasi loop(), program membaca nilai millis() ke variabel currentMillis. Nilai ini dibandingkan dengan previousMillis yang menyimpan waktu terakhir LED berubah. Jika selisih keduanya sudah mencapai atau melampaui 1000 ms, LED di-toggle dan previousMillis diperbarui dengan nilai currentMillis terkini. Proses ini berulang terus-menerus tanpa memblokir eksekusi.

2. Perbedaan delay() dan millis(): delay(n) menghentikan seluruh eksekusi program selama n milidetik, sementara millis() hanya membaca nilai timer tanpa menghentikan apapun. Dengan delay(), tidak ada kode lain yang dapat berjalan selama periode tunggu. Dengan millis(), program terus berjalan dan hanya mengeksekusi aksi tertentu ketika interval telah terpenuhi.

3. Alasan millis() disebut non-blocking: Karena fungsi ini tidak menghentikan eksekusi program. Program terus berjalan melalui loop() dengan normal; kondisi waktu hanya dicek, bukan ditunggu. Akibatnya, tugas-tugas lain dalam loop() tetap dieksekusi pada setiap iterasi.

4. Modifikasi dua LED berbeda interval: Untuk menjalankan dua LED dengan interval berbeda (LED1 = 1000 ms, LED2 = 500 ms), digunakan dua set variabel previousMillis secara terpisah (previousMillis1 dan previousMillis2) dengan interval masing-masing. Pada setiap iterasi loop(), kedua kondisi diperiksa secara independen. LED1 berubah setiap 1000 ms dan LED2 setiap 500 ms tanpa saling mengganggu, mendemonstrasikan konsep multitasking sederhana berbasis millis().

5. JAWABAN PERTANYAAN PRAKTIKUM

1. Keuntungan interrupt dibanding polling: Interrupt jauh lebih efisien dalam penggunaan CPU karena mikrokontroler hanya merespons ketika kejadian benar-benar terjadi, bukan terus-menerus memeriksa kondisi. Respons interrupt juga lebih cepat karena penanganan langsung terpicu oleh hardware, sementara polling bergantung pada frekuensi pengecekan dalam loop. Selain itu, interrupt membebaskan CPU untuk menjalankan tugas lain di antara kejadian [1][3].

2. Pentingnya timer dalam sistem embedded dan real-time: Timer menyediakan referensi waktu yang akurat dan independen dari beban kerja CPU. Pada sistem real-time, ketepatan timing sangat kritis untuk kendali motor, komunikasi serial, pembangkitan PWM, dan penjadwalan tugas. Timer memungkinkan sistem melakukan pengukuran waktu, menghasilkan sinyal periodik, dan menerapkan deadline eksekusi tanpa overhead polling yang boros [2][5].

3. Alur kerja sistem dengan interrupt dan timer digabung: Program utama berjalan normal dalam loop(). Timer secara periodik menghasilkan interrupt untuk menjalankan tugas terjadwal seperti pembacaan sensor atau pembaruan tampilan. Secara bersamaan, external interrupt dapat terpicu kapan saja oleh kejadian eksternal seperti penekanan tombol. Kedua jenis interrupt memiliki prioritas yang dapat dikonfigurasi; setelah masing-masing ISR selesai, kontrol dikembalikan ke titik eksekusi sebelumnya. Hasilnya adalah sistem yang responsif terhadap event eksternal sekaligus menjaga ketepatan timing periodik [1][2].

4. Dampak ISR yang terlalu panjang atau kompleks: ISR yang panjang menyebabkan program utama tertangguhkan dalam waktu lama, meningkatkan latency respons sistem. Interrupt lain yang terjadi saat ISR berjalan dapat terlewat atau tertunda. Jika menggunakan fungsi yang bergantung pada interrupt lain (seperti `delay()` atau `Serial.print()`), sistem dapat mengalami deadlock atau hang. Pada sistem real-time, ISR panjang dapat menyebabkan kegagalan deadline yang berdampak pada keselamatan sistem [1][4].

6. KESIMPULAN

Praktikum modul 6 berhasil mendemonstrasikan implementasi interrupt dan timer pada Arduino Uno melalui dua percobaan. Pada percobaan 6A, external interrupt berhasil mengubah kondisi LED secara langsung dan responsif saat tombol ditekan, tanpa memerlukan polling dalam loop() utama. ISR yang ringkas dengan variabel volatile memastikan

komunikasi data yang benar antara ISR dan program utama.

Pada percobaan 6B, metode `millis()` terbukti efektif sebagai pengganti `delay()` dalam implementasi LED blinking periodik. Sistem tetap responsif dan dapat menjalankan tugas lain secara bersamaan, mendemonstrasikan konsep multitasking sederhana pada embedded system. Pemahaman terhadap kedua mekanisme ini merupakan fondasi penting dalam pengembangan sistem embedded yang efisien dan responsif, terutama untuk aplikasi IoT, robotika, dan sistem monitoring real-time.

DAFTAR PUSTAKA

- [1] M. A. Mazidi, S. Chen, dan E. Ghaemi, AVR Microcontroller and Embedded Systems Using Assembly and C, 2nd ed., Pearson, New Jersey, 2011.
- [2] H. Monk, Programming Arduino: Getting Started with Sketches, 2nd ed., McGraw-Hill Education, New York, 2016.
- [3] A. Kadir, Pemrograman Arduino dan Android Menggunakan App Inventor, Elex Media Komputindo, Jakarta, 2016.
- [4] <https://www.arduino.cc/reference/en/language/functions/external-interrupts/attachinterrupt/>, diakses 19 Mei 2026, pukul 10.00 WIB.
- [5] <https://www.arduino.cc/en/Tutorial/BuiltInExamples/BlinkWithoutDelay>, diakses 19 Mei 2026, pukul 10.30 WIB.